Final Project: Design Your Own ISA

EECE.4821/5821 Computer Architecture

Released: **11/06/2026** Final due: **12/09/2026, 3:00 pm Eastern**

Individual assignment 100 points (4821) 120 points (5821) +5 extra credit

The idea

All semester you have used someone else's instruction set. In Project 1 you took a binary apart, and in Project 2 you watched RISC-V move through a pipeline. In the final project you design your *own* instruction set architecture (ISA) from a blank page, then build a simulator that runs programs on it, one pipeline stage at a time.

You choose almost everything: the word size, the instruction width, how many registers there are, how many instructions there are, how they are encoded, and how many pipeline stages there are. There is one goal behind all of these choices:

The design goal

Make your ISA as **minimal** as you can while it can still run every benchmark program in Section 2. Every instruction, register and bit must earn its place. A smaller ISA is not automatically better, though. If removing an instruction makes the benchmarks three times longer, you have to argue that the trade was worth it.

Learning objectives

1. Turn a set of program requirements into an instruction set: operations, operands, addressing modes and control flow.
2. Design instruction formats and an encoding, and justify the bit budget.
3. Weigh ISA size against program size, instruction count and hardware complexity.
4. Design a pipelined datapath for your ISA and draw it with its pipeline registers, control signals and multiplexers.
5. Implement a cycle-by-cycle pipeline simulator and use it to measure cycles and CPI.
6. *(5821)* Detect and resolve data and control hazards in hardware.

Schedule at a glance

Milestone	What you submit	Due	Points
M1. Requirements	1–2 page proposal for approval (pass/revise)	**11/13/2026, 11:59 pm**	10
M2. ISA card	Reference card, encodings, one hand-assembled benchmark	**11/25/2026, 11:59 pm**	20
Final	Simulator, write-up, two slides, presentation	**12/09/2026, 3:00 pm Eastern**	70 (+20)

1 Ground rules for your ISA

The one required instruction

Your ISA must have a `HALT` instruction. When `HALT` retires, the simulator stops. Everything else is your choice.

Everything else is open

Word size, instruction width (fixed or variable), number and kind of registers (general-purpose, accumulator, stack, none), memory model, addressing modes, condition codes or compare-and-branch, how immediates are encoded. Every one of these is a design decision you must state and justify.

No multiply or divide instructions

The benchmarks are about building operations from simpler ones. You may not add a `MUL`, `DIV` or any instruction that does a whole benchmark in one step.

Your own design

You may borrow ideas from RISC-V, MIPS, ARM, LC-3 or anything else, and say where they came from. You may not copy an existing ISA, or a subset of one, with its encoding.

Narrow words are allowed

If your data word is narrower than a benchmark's result, you need multi-word arithmetic (a carry). That is a legitimate trade-off; account for its cost in your write-up.

2 Benchmark programs

Your ISA must run all four benchmarks correctly on your simulator. Operand values will be changed when grading, within the ranges given, so do not hard-code answers. Document where each program's inputs are stored and where it leaves its result (register or memory address).

Benchmark	Task	Exercises
B1. Array sum	Sum 8 unsigned values stored in data memory. Each value is 0–15, so the sum is at most 120. Test data: `3, 14, 7, 0, 15, 9, 2, 11` $\rightarrow 61$.	loop, memory access, addition
B2. Multiply	Multiply two unsigned values $a, b \in [0, 15]$ by shift-and-add, so the product is at most 225. Test data: $13 \times 11 = 143$.	logical shifts, bit tests, branches
B3. Function call	Compute $a{\cdot}b{+}c{\cdot}d$ with $a, b, c, d \in [0, 11]$, so the result is at most 242, by *calling B2 as a subroutine twice* from two different call sites. Test data: $6{\cdot}7{+}5{\cdot}9 = 87$.	call, return, saving state
B4. Your choice	A program you choose that shows off a strength of your ISA, or honestly shows a weakness. Explain what it demonstrates.	your design

Between them, the benchmarks require your ISA to be able to add, shift logically left and right, read and write memory, branch on a condition, and call and return from a subroutine. How it does each of these, whether with a dedicated instruction or a sequence of simpler ones, is up to you.

3 Milestone 1: Requirements 10 pts, pass/revise

Due **11/13/2026, 11:59 pm**. Submit `<lastname>_FP_M1.pdf`, 1–2 pages.

This is your proposal. I read it and either approve it or return it for revision. An approved proposal earns the full 10 points. If it is returned, you have **3 days** from the time I return it to resubmit. Do not build your simulator around a design that has not been approved.

Your proposal must cover:

M1.1 Design philosophy. In 2–3 sentences, the idea behind your ISA. For example: "an 8-bit accumulator machine with the fewest possible opcodes," or "a 12-bit load/store RISC with four registers."

M1.2 Planned parameters. Word size, instruction width, number of registers, memory model (separate or unified instruction and data memory, address width), and a first estimate of the number of instructions.

M1.3 Planned instruction list. Each instruction with a one-line description. Encodings are not needed yet.

M1.4 Benchmark plan. For each of B1–B3, a few sentences or pseudocode showing how your instructions will do it. Name B4.

M1.5 Pipeline plan. The number of stages and what each stage does. You need at least **3 stages**.

M1.6 Simulator plan. The language you will use, and whether you plan to attempt the assembler extra credit.

4 Milestone 2: The ISA reference card 20 pts

Due **11/25/2026, 11:59 pm**. Submit `<lastname>_FP_M2.pdf`.

Make a reference card for your ISA in the style of the RISC-V "green card": everything a programmer needs to write and encode programs, in at most **2 pages**. After the card, add the hand-assembled benchmark and the pipeline sketch.

M2.1 (8 pts) The card. It must contain:

- **Registers:** names, numbers, width, and any special roles (zero register, return address, stack pointer, flags). Include the PC.
- **Instruction formats:** a bit-field diagram for each format, with field names and bit positions.
- **Instruction table:** for every instruction, give the mnemonic, the assembly syntax, its format, its opcode (and any function bits) in binary, and its operation in register-transfer notation (for example `R[rd] ← R[rs] + sext(imm)`; `PC ← PC + 1`).
- **Immediates and addressing:** immediate widths, signed or unsigned, how they are extended, and every addressing mode.
- **Branch and jump targets:** absolute or PC-relative, and the units of the offset (bytes or words).
- **Memory map:** address width, word or byte addressing, and where code, data and the stack (if any) go.

- **Calling convention:** how B3 passes arguments, returns a result and saves the return address.

M2.2 (4 pts) Consistency. The encodings must be unambiguous: no two instructions may share an encoding, and every field must fit in its bits. I will spot-check by decoding some of your instructions by hand.

M2.3 (4 pts) Hand-assembled B2. The multiply benchmark in your assembly language, with the machine code of each instruction in binary and hex beside it, and its static size in instructions and bits.

M2.4 (4 pts) Preliminary pipeline sketch. A block diagram of your stages, the pipeline registers between them, and the main units (register file, ALU, memory). This does not have to be final.

What a format diagram can look like (example only, do not copy)

	15 12	11 9	8 6	5 0
R-type	opcode	rd	rs	funct
I-type	opcode	rd	rs	imm[5:0]

5 Final: The simulator 35 pts

Write a simulator for your ISA in any language you like (Python, C, C++, Java, Rust, ...). It must be your own code: do not build on top of an existing simulator framework. It has to run from the command line with the commands given in your `README`.

Required behaviour

S1. Program loading. Read a program from a text file of machine code, one instruction per line in hex or binary, with comments allowed. Initial data memory comes from the same file (a data section) or from a second file. Document the format.

S2. Functional mode. Execute one whole instruction at a time, with no pipeline. This is your reference: it tells you what the right answer is.

S3. Pipeline mode. Model your pipeline cycle by cycle (see the box below). Each simulated clock cycle, every instruction in flight moves one stage forward through the pipeline registers.

S4. Output. At `HALT`, print the final registers, the data memory that the program uses, the cycle count, the instructions retired and the CPI.

S5. Trace. A flag (for example `-trace`) that prints a pipeline diagram: one row per cycle (or per instruction) showing which instruction is in each stage, with bubbles and stalls marked, like the diagram in Ripes.

S6. Self-check. For every benchmark, functional mode and pipeline mode must end with the same registers and memory.

How to simulate a pipeline

Keep one record (a `struct`, class or dictionary) for each pipeline register, for example `IF/ID`, `ID/EX`, `EX/MEM` and `MEM/WB` for a 5-stage design. Each record holds the instruction and every value and control signal that the later stages need. Each simulated cycle then works like a clock edge:

1. Compute every stage from the *current* pipeline registers: WB writes the register file, MEM reads or writes memory, EX runs the ALU, ID decodes and reads registers, IF fetches. Doing the stages in the order WB, MEM, EX, ID, IF makes the register file written in the first half of the cycle and read in the second, just as in our textbook pipeline.
2. Store the results in a set of *next* pipeline registers.
3. Copy next into current all at once. That is the clock edge.

A bubble is a pipeline register whose control signals are all "do nothing." A stall holds the PC and the earlier pipeline registers and injects a bubble. A flush turns instructions that are already in flight into bubbles. The cycle count is the number of times you went round this loop.

Hazards in the undergraduate pipeline

EECE.4821 students do **not** build hazard hardware. Your pipeline behaves like Ripes' `RV32_5S_NO_FW_HZ` in Project 2. Programs stay correct because the program itself avoids hazards:

- **Data hazards:** put `NOP`s (or independent instructions) between a producer and its consumer. State the rule: how many instructions must separate them in *your* pipeline, and why.
- **Control hazards:** define in your ISA what happens to the instructions fetched behind a branch or jump. The usual answer without flush hardware is *branch delay slots*: they always execute. Say how many there are, and fill them with `NOP`s or useful work.

Your benchmark programs for pipeline mode include these `NOP`s. Report their cost.

Run the benchmarks early

A pipeline that hides a hazard bug still gives the right answer in functional mode, so S6 is how you find it. Compare the two modes on every benchmark as soon as your pipeline runs, not the night before the deadline.

6 Final: The write-up 20 pts

Submit `writeup.pdf`, about 8–12 pages plus appendices. It must contain:

W1. Design overview and decisions. Your design philosophy, and every major decision with the alternative you rejected and why: word size, instruction width, registers, formats, addressing modes, branch style, and pipeline depth.

W2. The final ISA reference card, updated from M2, and a list of what changed since M2 and why.

W3. Minimality analysis. For each instruction: why it is needed and what it would cost to remove it (which benchmarks get longer, and by how much). Then fill in this scorecard:

ISA parameter	Value
Word size (bits)	
Instruction width (bits)	
Number of instructions	
Number of formats	
Visible registers	

Benchmark	Static instr.	Static bits	Dynamic instr.	Cycles (pipeline)
B1				
B2				
B3				
B4				

W4. Benchmark implementations. The assembly listing of each benchmark with comments, its machine code, and the simulator output for the test data. Show at least one more input for B1–B3 beyond the test data.

W5. Full pipeline design diagram. A complete datapath drawn with a tool, not a photo of a sketch: every stage, every pipeline register, the register file, ALU, memories, every multiplexer, and the control unit with the signals it sends down the pipeline. Label each control signal and list its values for every instruction in a table, as in Project 2 Part A. (5821: include the hazard detection unit, the forwarding paths and the flush logic.)

W6. Simulator design. How the code is organized, how it models the pipeline registers and the clock edge, and how you tested it.

W7. Hazard analysis. The `NOP` rules and delay-slot cost (4821), or the hazard control results (5821, below).

W8. Reflection. What you would change if you designed the ISA again.

7 Final: The presentation 15 pts

Submit `slides.pdf`, **at most 2 slides** (a title slide is not needed and does not count). You will present on the last day of class, 12/09, in about **4 minutes** including questions.

- **Slide 1: your ISA at a glance.** The design philosophy, the key parameters, and the instruction formats.
- **Slide 2: the result.** Your pipeline and the benchmark scorecard. 5821 students also show the effect of hazard control.

EECE.5821 only: Hazard control **+20 pts**

Add hazard control hardware to your pipeline simulator, so that the benchmarks run correctly *without* any `NOP`s or delay slots.

G1. (6 pts) Hazard detection and stalls. Detect read-after-write dependences and stall the pipeline until the value is available.

G2. (6 pts) Forwarding. Add forwarding (bypass) paths so that most dependences no longer stall. Stall only where forwarding cannot help, such as a load followed by its use, if that hazard exists in your pipeline.

G3. (4 pts) Control hazards. Flush the wrong-path instructions on a taken branch or jump, or implement a simple predictor. Your ISA must no longer need delay slots.

G4. (4 pts) Evidence and analysis. Add a flag that switches hazard control off, so the same program can be compared with and without it. Write `hazards.prog`, a test program with at least one of each hazard type that your pipeline has, and show the annotated trace for each. In the write-up, give a table of cycles and CPI for B1–B4: (a) with `NOP`s and no hazard hardware, (b) stalls only, (c) stalls and forwarding, (d) the full design. Explain where the cycles went.

If your pipeline design makes a hazard impossible (for example, a 3-stage pipeline without a separate memory stage), say so and prove it with a trace. Do not invent the hazard. A shallower pipeline has fewer hazards, but you must defend its longer clock period in W1.

Extra credit: an assembler **+5 pts**

Write an assembler that turns your assembly language into the machine-code file your simulator loads. It must support labels for branch targets and data, report errors with line numbers, and assemble all four benchmarks. You may then use it instead of hand-encoding, but you still hand-assemble B2 for M2.

8 Final deliverables and submission

Due **12/09/2026, 3:00 pm Eastern**. Submit one `<lastname>_FP.zip` (for example: `boewe_FP.zip`) containing:

`README.md`	Build and run commands, the program file format, and the exact command to reproduce each benchmark result
`src/`	Simulator source (and assembler, if any)
`programs/`	B1–B4 as assembly (`.asm`) and machine code; for 5821, also `hazards.prog` and the versions without `NOP`s
`output/`	The simulator output for each benchmark, in both modes
`writeup.pdf`	The write-up
`slides.pdf`	Your two slides

The 3:00 pm deadline is firm

The final is due at **3:00 pm Eastern** on 12/09, not at midnight, because presentations are that day. I will run your simulator on all four benchmarks with new input values, using only the commands in your `README`. If it does not build or run from those commands, the simulator correctness points are lost.

Point summary

Component	Points	Graded on
M1. Requirements (pass/revise)	10	approved proposal
M2. ISA reference card	20	card 8, consistency 4, hand-assembled B2 4, pipeline sketch 4
Simulator	35	benchmarks correct in both modes 15, pipeline model and trace 12, README, code quality and self-check 8
Write-up	20	decisions and minimality 7, pipeline diagram 6, benchmarks 4, simulator and hazard analysis 3
Presentation	15	clarity 5, design rationale 5, results 5
Total	**100**	
EECE.5821 hazard control	+20	out of 120 for 5821
Extra credit: assembler	+5	both sections